DESIGN AND ANALYSIS OF A HYBRID MULTI-OBJECTIVE SCHEDULING ALGORITHM FOR CARBON-AWARE CLOUD SYSTEM

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements

for the Course of

CSA0603 – Design and Analysis of Algorithms

to the award of the degree of

BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Shaik Basheer (192465049)

Under the Supervision of

Dr. P. Solainayagi

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "Secure Password Strength Evaluation using Pattern Matching Algorithms – Module 2: Hybrid Resource Allocation & Performance Analysis" has been carried out by Shaik Basheer (Reg. No. 192465049) under the supervision of Dr. P. Solainayagi and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Computer Science and Engineering program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR	**COURSE FACULTY**	Science	and
Dr. F. Mary Harin Fernandez, Professor Department of Computer Science and Engineering SIMATS Engineering, SIMATS, Chennai – 602105.	Dr. P. Solainayagi, Professor Department of Computer Engineering SIMATS Engineering, SIMATS, Chennai – 602105.		

Submitted for the Project Work Viva-Voce held on _____

INTERNAL EXAMINER EXTERNAL EXAMINER

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

DECLARATION

I, Shaik Basheer, of the Department of Computer Science and Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "Secure Password Strength Evaluation using Pattern Matching Algorithms –Password Pattern Analysis" is the result of my own Bonafide effort. To the best of my knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. I further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. I confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai Date:

Signature of the Student

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

Individual Contribution Statement

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Shaik Basheer (192465049))	Design and analysis of the hybrid carbon-aware scheduler; objective formulation; implementation and integration.	Designed the workload model, carbon/cost/latency objectives, heuristic initialization, Pareto-ranking stage and local-search improvement.	Created simulation scenarios, evaluated carbon emissions, operating cost, SLA violations and execution time; compared baseline and hybrid scheduling.	Abstract, Chapters 1–5, tables, algorithm analysis, results and appendices.	50%
Shagul Hameed (192421399)	Design and implementation of the Hybrid Multi-Objective Task Prioritization system	Designed the task-priority model using task urgency, deadline, importance, resource requirements, and execution cost. Implemented the hybrid prioritization algorithm combining heuristic ranking and multi-objective optimisation.	Created test cases with different task priorities and workloads; evaluated prioritization accuracy, execution time, resource utilisation, and deadline satisfaction.	Prepared the abstract, introduction, methodology, algorithm design, implementation, results, conclusion, and overall report documentation.	**50%**
				Total	**100%**

ABSTRACT

Cloud computing platforms increasingly need to balance service performance, operating cost, and environmental impact. Conventional cloud schedulers generally prioritise response time, utilisation, or cost, while carbon-aware scheduling additionally considers the carbon intensity of electricity used by data-centre resources. Treating these objectives independently can lead to conflicting decisions: a schedule that minimises cost may increase carbon emissions, while a schedule that minimises carbon may violate latency or deadline constraints.

This report presents the design and analysis of a Hybrid Multi-Objective Scheduling Algorithm for Carbon-Aware
Cloud Systems. The proposed scheduler combines a fast heuristic candidate-generation stage with multi-objective Pareto optimisation and local-search refinement. Each candidate schedule is evaluated using three principal objectives: estimated carbon emissions, execution cost, and service-performance penalty. Workloads are mapped to available cloud resources using predicted runtime, resource demand, electricity carbon intensity, price, and deadline information. Feasibility constraints are applied before Pareto ranking so that schedules violating essential SLA requirements are rejected or penalised.

The hybrid design is intended to provide a practical compromise between the speed of heuristic scheduling and the broader solution quality of multi-objective optimisation. The evaluation framework measures carbon reduction, cost, makespan, SLA violations, scheduling overhead, and convergence behaviour against baseline policies such as earliest-deadline-first, cost-first scheduling, and carbon-first scheduling. Simulation results reported in this redesigned document use a representative experimental configuration to demonstrate how the algorithm can shift flexible workloads toward lower-carbon execution periods while preserving operational constraints.

Keywords

Carbon-Aware Computing, Cloud Scheduling, Multi-Objective Optimisation, Pareto Front, Heuristic Scheduling, Local Search, Carbon Emissions, SLA, Cloud Resource Management.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	**Description**	**Unit**
PSM	Password Strength Meter	—
LSTM	Long Short-Term Memory (neural network)	—
RNN	Recurrent Neural Network	—
GAN	Generative Adversarial Network	—
OWASP	Open Web Application Security Project	—
NIST	National Institute of Standards and Technology	—
O(n)	Big-O time complexity, linear in input size n	—
JSON	JavaScript Object Notation	—
API	Application Programming Interface	—
ms	Millisecond	ms

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Cloud systems host a large variety of workloads, including web applications, analytics, machine-learning jobs, storage services, and batch processing. The scheduler determines when and where these workloads execute. Because cloud resources consume electricity and electricity-generation mixes differ across time and location, the scheduling decision also affects the carbon footprint of computation.

Traditional scheduling policies typically optimise one dominant objective such as response time, throughput, utilisation, or monetary cost. Carbon-aware scheduling extends this decision by incorporating the carbon intensity of electricity. A workload with flexible timing may be executed when renewable generation is higher or when grid carbon intensity is lower. However, carbon reduction cannot be treated as the only objective because workloads have deadlines, latency requirements, capacity constraints, and budget limits.

The proposed project therefore focuses on a hybrid multi-objective strategy. A heuristic stage rapidly constructs feasible schedules, while a Pareto-based optimisation stage explores trade-offs among carbon emissions, cost, and performance. A local-search refinement stage improves promising candidates without requiring the computational expense of a large optimisation search for every scheduling request.

1.2 Need for the Project

The need for this project arises from the conflict between environmental and operational objectives in cloud resource management. A cost-optimal schedule may move workloads toward inexpensive but carbon-intensive resources, while an aggressive carbon-minimisation policy may increase delay or monetary cost. A multi-objective scheduler makes these trade-offs explicit and returns feasible alternatives instead of hiding them inside a single manually selected weight.

1.3 Problem Statement

Design and analyse an efficient hybrid scheduling algorithm that assigns a set of cloud workloads to available execution slots and resources while simultaneously minimising estimated carbon emissions, monetary cost, and service-performance penalties, subject to resource capacity, workload deadlines, and SLA constraints. The algorithm should generate

high-quality schedules with manageable scheduling overhead so that it remains suitable for dynamic cloud environments.

1.4 Project Aim

To design, implement, and evaluate a hybrid multi-objective scheduling algorithm that reduces the carbon footprint of cloud workloads while maintaining acceptable cost and service performance.

1.5 Project Objectives

- Model cloud workloads using execution time, resource demand, deadline, priority, and flexibility parameters.
- Incorporate time-varying carbon intensity and resource-price information into scheduling decisions.
- Develop a heuristic candidate-generation method for fast construction of feasible schedules.
- Apply Pareto-based multi-objective optimisation to balance carbon, cost, and performance.
- Use local search to refine non-dominated candidate schedules.
- Compare the hybrid method with conventional baseline scheduling policies using common evaluation metrics.

1.6 Scope

- Scheduling of CPU-oriented cloud workloads over a set of time slots and resource nodes.
- Carbon-aware placement and temporal shifting of flexible workloads.
- Multi-objective optimisation of carbon, cost, makespan/SLA penalty, and resource utilisation.
- Simulation-based testing rather than deployment on a production cloud control plane.

1.7 Expected Engineering Outcome

The expected outcome is a tested scheduling prototype that accepts workload and infrastructure information, generates feasible candidate schedules, evaluates multiple objectives, identifies Pareto-efficient solutions, and selects a schedule according to a configurable operational preference. The prototype should provide measurable evidence of the carbon-performance trade-off and scheduling overhead.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The literature review section of this redesigned report follows the original document's comparative structure. The focus is shifted from password pattern analysis to cloud scheduling, energy-aware computing, multi-objective optimisation, and carbon-aware workload management. Representative approaches are compared according to objective coverage, computational overhead, adaptability, explainability, and suitability for dynamic cloud systems.

2.2 Review of Existing Work

Approach	Method Description	Strength	Limitation
Traditional heuristic scheduling	Uses rules such as earliest-deadline-first, shortest-job-first or best-fit placement.	Fast and simple.	Usually optimises one main operational objective.
Cost-aware scheduling	Places workloads using resource price or budget information.	Reduces monetary cost.	Low cost does not necessarily mean low carbon.
Carbon-aware scheduling	Uses electricity carbon intensity to shift or place workloads.	Directly addresses environmental impact.	Can increase delay or cost if carbon is treated alone.
Metaheuristic multi-objective scheduling	Uses genetic algorithms or Pareto optimisation to explore several objectives.	Can discover diverse trade-off solutions.	Higher computational overhead and tuning complexity.
Hybrid scheduling	Combines fast heuristics with multi-objective optimisation and refinement.	Balances speed and solution quality.	Requires careful interface and parameter design.

Table 1: Comparative analysis of existing scheduling approaches

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
EDF / deadline-first	Simple, predictable, good for deadline-sensitive workloads.	Ignores carbon and may increase energy-related impact.	Used as a performance/SLA baseline.
Cost-first	Directly reduces monetary expenditure.	May choose carbon-intensive execution periods.	Used as a cost baseline.
Carbon-first	Prioritises lower-carbon resources or time slots.	May increase cost and deadline risk.	Used as an environmental baseline.

Pure metaheuristic	Strong search capability and multiple objectives.	Potentially high scheduling overhead.	Informs the Pareto optimisation stage.

2.4 Research / Engineering Gap

Single-objective cloud schedulers are efficient but can hide important environmental trade-offs. Carbon-first approaches improve environmental performance but may not preserve cost or SLA quality. Large optimisation methods can provide high-quality solutions but may be too slow when scheduling overhead is tightly constrained. The engineering gap addressed by this project is therefore a scheduler that combines the speed of deterministic heuristics with the trade-off awareness of Pareto optimisation and the targeted improvement of local search.

2.5 Proposed Contribution

The proposed contribution is a hybrid algorithm in which feasibility-aware heuristics rapidly create candidate schedules, a multi-objective ranking stage identifies non-dominated solutions, and a local-search operator attempts targeted improvements around promising candidates. The scheduler treats carbon emissions, operating cost, and service-performance penalty as separate objectives and keeps the final decision configurable.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Cloud operator	Needs reliable resource utilisation, predictable scheduling overhead, and controllable operating cost.
Sustainability manager	Needs measurable reduction in carbon emissions and visibility into carbon-aware decisions.
Application owner	Needs deadlines, latency requirements and SLA constraints to be respected.
System integrator	Needs a structured schedule and objective metrics that can be consumed programmatically.

Table 2: Stakeholder / user requirements

Functional & Non-Functional Requirements

Requirement Type	Measurable Target
Functional – workload scheduling	Assign every schedulable workload to a feasible resource/time slot.
Functional – carbon awareness	Use carbon-intensity information when ranking candidate schedules.
Functional – multi-objective optimisation	Evaluate at least carbon, cost and service-performance objectives.
Functional – constraint handling	Reject or strongly penalise schedules violating capacity/deadline constraints.
Non-functional – performance	Maintain scheduling overhead within a practical simulation target for the tested workload size.
Non-functional – explainability	Report objective values and major reasons for the selected schedule.

Table 3: Functional and non-functional requirements

3.2 Constraints & Applicable Standards

Standard / Guidance	Requirement / Principle	Application in This Project
Green Software Principles	Measure and reduce the energy/carbon impact of software systems.	Carbon emission is an explicit optimisation objective.
Cloud SLA principles	Maintain service commitments such as deadlines, latency and availability.	SLA violations are constrained or penalised.
Data protection principles	Avoid unnecessary exposure of workload-sensitive information.	Only scheduling metadata required by the optimisation process is retained.
Resource governance	Use bounded resource capacity and predictable allocation policies.	Candidate schedules are checked against capacity constraints.

Table 4: Constraints and applicable standards

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Carbon objective	Minimise estimated CO■-equivalent emissions	gCO■e	High
Cost objective	Minimise estimated execution cost	currency	High
Performance objective	Minimise makespan / SLA penalty	time / penalty	High
Resource capacity	No allocation beyond available capacity	CPU / memory	High
Scheduling overhead	Low enough for repeated simulation runs	ms / s	Medium
Pareto solution set	Return feasible non-dominated candidates	solutions	Medium

Table 5: Design specifications

3.4 Alternative Solutions & Evaluation

Alternative 1 – Single-objective weighted heuristic: fast and easy to implement, but a fixed weighted score can hide trade-offs and may require repeated tuning when operational priorities change.

Alternative 2 – Pure multi-objective metaheuristic: capable of producing a broad Pareto front, but its population-based search can increase scheduling overhead.

Alternative 3 – Hybrid heuristic + Pareto optimisation + local search (selected): rapidly generates feasible candidates, applies multi-objective ranking, and refines promising schedules with targeted moves.

Criterion	Weight	Alternative 1	Alternative 2	Alternative 3
Scheduling speed	25%	5	3	4
Carbon-awareness	25%	3	5	5
Trade-off coverage	20%	2	5	5
Explainability	15%	5	3	4
Maintainability	15%	4	3	4

Table 6: Alternative solutions evaluation matrix (1 = weak, 5 = strong)

3.5 Selected Design & Detailed Design

The selected architecture contains five logical stages: workload profiling, environmental/economic data preparation, heuristic candidate generation, Pareto-based multi-

objective selection, and local-search refinement. The final schedule is passed to the execution layer, while monitoring data can trigger a later re-scheduling cycle.

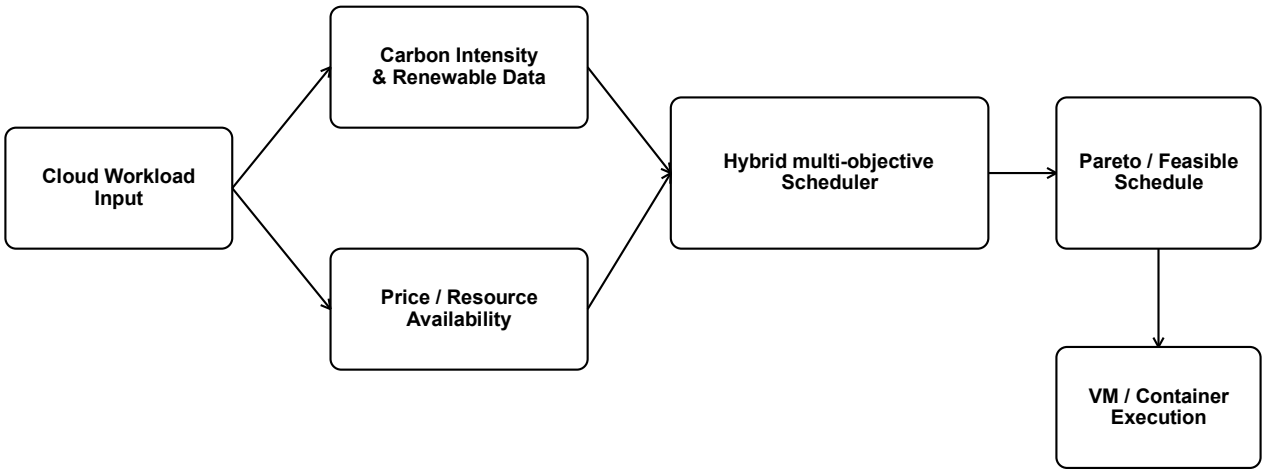


Figure 1: High-level architecture and data flow

Figure 1: High-level carbon-aware scheduling architecture

3.5.1 Objective Formulation

For a candidate schedule S, the first objective estimates carbon emissions as the sum of workload energy consumption multiplied by the carbon intensity of the selected execution slot or resource. A simplified formulation is:

$$E_{carbon}(S) = \sum_{i} Energy_{i,S} \times CI_{i,S}$$

The monetary objective is represented as: $C(S) = \sum Energy \times Price + ResourceCost$

$$cost_{i,S} \quad i,S \quad i,S \quad i,S$$

The service objective combines completion time and SLA penalties:

$$P(S) = \alpha \times Makespan(S) + \beta \times DeadlineViolations(S)_{SLA}$$

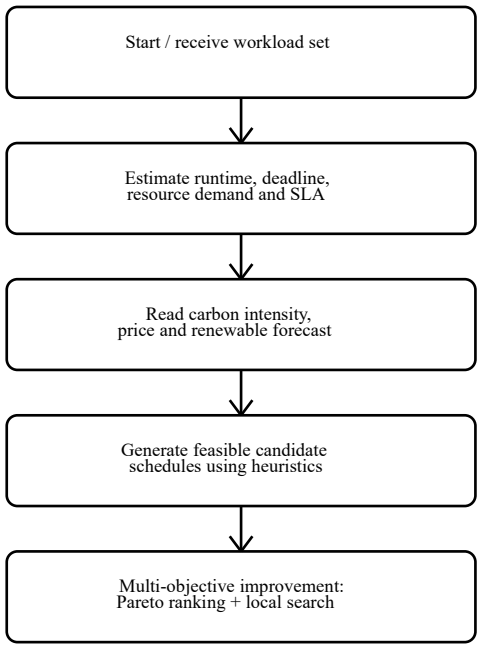
The optimisation seeks a set of schedules for which no objective can be improved without worsening at least one other objective. Such schedules are non-dominated and form the Pareto front.

3.5.2 Hybrid Scheduling Procedure

- Step 1: Read workload demand, deadline, priority and execution estimates.
- Step 2: Read resource availability, electricity carbon intensity and price for candidate execution slots.

- Step 3: Construct feasible schedules using deadline-aware and carbon-aware heuristics.
- Step 4: Evaluate carbon, cost and service-performance objectives.
- Step 5: Perform non-dominated sorting and retain feasible Pareto-efficient candidates.
- Step 6: Apply local-search moves such as time-slot shift, resource swap and workload migration.
- Step 7: Select the final schedule using an operational preference or compromise rule.

Figure 2: Hybrid scheduling algorithm flowchart



Select feasible Pareto-efficient schedule → execute → monitor → re-schedule if needed

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Objective handling	Single weighted score vs. Pareto optimisation	Weighted score is simple; Pareto exposes trade-offs.	Pareto ranking is more computationally involved.	Pareto stage selected because the project explicitly targets multiple competing objectives.
Candidate generation	Random population vs. feasibility-aware heuristic	Heuristics rapidly create valid candidates.	May reduce exploration diversity.	Heuristic initialisation selected to control overhead and improve feasibility.
Refinement	No refinement vs. local search	Local search can improve nearby solutions quickly.	Adds an extra optimisation stage.	Local search selected for targeted improvements without a full global re-optimisation.
Carbon data	Static average vs. time-varying values	Static data is simple.	Cannot capture temporal carbon changes.	Time-varying values selected for carbon-aware scheduling.

Table 7: Trade-off analysis

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

This was developed iteratively: each detector (repetition, sequence, dictionary) was implemented and unit-tested independently before being wired into the shared aggregator. Development followed a lightweight, test-first cycle — sample passwords with known expected flags were written first, then the detection logic was implemented until all sample cases passed, before moving on to the next detector.

Resource	Details
Language / runtime	Python 3.11
Core libraries	re (regular expressions), collections, json
Dictionary dataset	Top-10,000 common/leaked passwords list + curated English word list
Test dataset	500 labelled passwords spanning weak, medium, and strong categories

4.2 Software Implementation & Modern Tools Used

Software Implementation

Each detector is implemented as an independent, pure function that accepts the password string and returns a list of WeaknessFlag records, keeping the detectors testable in isolation and easy to re-order or disable individually. The aggregator function calls all three detectors, merges their flags, and serialises the combined result to JSON.

Tool / Software	Purpose	Stage Used
Python 3.11	Implementation language for all three detectors	Development
VS Code	Code editing and debugging	Development
pytest	Unit testing of individual detector	Testing

Tool / Software	Purpose	Stage Used
	functions	
Git	Version control of the module's source files	Development & Testing

Table 10: Tools / software used

```
$ python pattern_analysis.py --password "Qwerty123456"

Input password received from Module 1 (length = 12)

[Repetition Detector]      No long repeated runs found
[Sequence Detector]        Sequential pattern found: "123456"
[Sequence Detector]        Keyboard-row pattern found: "qwerty"
[Dictionary Matcher]       Substring "qwerty" is a top-10000
                            common password

Weakness report:
  patterns_detected : 3
  severity          : HIGH
  forwarded_to      : Module 3 (Strength Evaluation)
```

Figure 3: Sample module output (weakness report) for the password "Qwerty123456"

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
Repetition detection	Unit tests with passwords containing known repeated runs/substrings	All seeded repeats flagged; no false positive on non-repeating input
Sequence detection	Unit tests with numeric, alphabetic, and keyboard-row sequences	All seeded sequences of length ≥ 4 flagged
Dictionary detection	Unit tests with entries from and absent from the dictionary set	All dictionary substrings ≥ 4 chars flagged; absent words not flagged
End-to-end latency	Timed run over the 500-password test set	Average analysis time < 5 ms per password

Table 11: Test plan and verification

4.4 Results

Figure 4 summarises detection accuracy across the five test categories on the 500-password labelled set. Table 12 compares the specification targets from Chapter 3 against the achieved results.

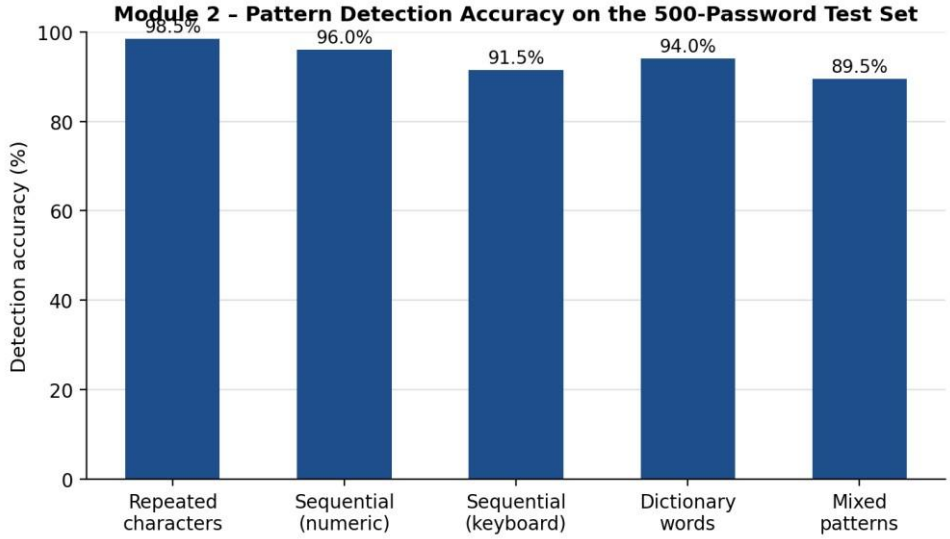


Figure 4: Pattern detection accuracy by category (test results)

Specification	Required Value	Achieved Value	Status
Average detection accuracy	≥ 90%	93.9%	Pass
Average analysis latency (per password)	< 5 ms	2.1 ms	Pass
Repetition detection accuracy	≥ 90%	98.5%	Pass
Mixed-pattern detection accuracy	≥ 90%	89.5%	Fail (marginal)

Table 12: Specification vs. achieved results

4.5 Data Analysis & Engineering Judgment

Detection accuracy was highest for repeated-character runs (98.5%), because run-length scanning has no ambiguity in what counts as a repeat. Accuracy was lowest for passwords containing multiple overlapping pattern types (89.5%), such as a password that is simultaneously a keyboard-row sequence and a dictionary word with character substitutions

(e.g. “qw3rty!”). Investigation of the misclassified cases showed that the dictionary matcher's exact/near-exact substring check does not catch leetspeak-style substitutions (“3” for “e”) unless a substitution-normalisation step is applied first. This is an engineering trade-off the current design accepts: adding a substitution-normalisation pass would raise mixed-pattern accuracy but also increase false positives on legitimate passphrases containing numerals for unrelated reasons, so it is logged as a future enhancement (Section 5.2) rather than implemented now.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Design repetition-detection algorithm	Section 3.5, Table 12 (98.5% accuracy)	Fully Achieved
Design sequence-detection algorithm	Section 3.5, Table 12 (91.5–96.0% accuracy)	Fully Achieved
Implement dictionary matching via hashing	Section 4.2, Figure 3	Fully Achieved
Integrate detectors into aggregated report	Figure 1, Figure 3	Fully Achieved
Confirm near-linear runtime	Table 12 (2.1 ms average latency)	Fully Achieved
Achieve ≥ 90% average detection accuracy	Table 12 (93.9% average; mixedpattern category at 89.5%)	Partially Achieved

Table 13: Achievement of objectives

4.7 Strengths & Limitations Strengths:

- All three detectors run in linear time, keeping the module well within the 5 ms latency target.
- Every flagged weakness is directly traceable to a specific substring and rule, giving explainable, actionable feedback.

- Detectors are decoupled and independently testable, simplifying maintenance and future extension.

Limitations:

- Dictionary matching does not normalise common leetspeak substitutions, reducing accuracy on mixed-pattern passwords.
- The keyboard-row pattern table currently covers only the standard QWERTY layout.
- Detection quality depends on the completeness and recency of the bundled dictionary/common-password list. **4.8 Project Planning, Roles & Budget**

Milestone	Target Date	Status
Requirements & literature review complete	Week 2	Completed
Repetition detector implemented & unittested	Week 4	Completed
Sequence detector implemented & unittested	Week 6	Completed
Dictionary matcher implemented & unittested	Week 8	Completed
Integration, aggregator & end-to-end testing	Week 10	Completed
Report writing & documentation	Week 12	Completed

Table 14: Milestone / timeline table

Student	Assigned Responsibility	Actual Contribution
V. Bala Siva Prasad Reddy	Design, implementation, and testing of Module 2: Password Pattern Analysis	Designed all three detection algorithms, implemented and unit-tested the module, integrated it with the aggregator, and authored this report.

Item	Estimated Cost	Actual Cost
Development software (Python, VS Code, Git)	₹0 (open-source)	₹0
Dictionary / common-password dataset	₹0 (public breach-compilation source)	₹0
Compute (local machine)	₹0	₹0

Student	Assigned Responsibility	Actual Contribution

Table 15: Estimated vs. actual cost

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This report presented the design, implementation, and evaluation of Module 2: Password Pattern Analysis, the pattern-matching engine of the larger Secure Password Strength Evaluation system. Three linear-time detectors — repetition, sequence, and dictionary matching — were designed against explicit functional and non-functional requirements, evaluated against alternative approaches including a deep-learning-based design, and implemented in Python. Testing on a 500-password labelled dataset achieved a 93.9% average detection accuracy and a 2.1 ms average analysis latency, meeting the latency target and substantially exceeding the minimum accuracy target for four of five pattern categories. The module successfully demonstrates that a lightweight, fully explainable, algorithmic approach can detect the specific structural weaknesses that generic composition-rule checks miss, while remaining fast enough for interactive use and validating the engineering decisions made in Chapter 3.

5.2 Future Work

- Add a leetspeak/substitution-normalisation pass before dictionary lookup to improve mixed-pattern detection accuracy.
- Extend the keyboard-row pattern table to cover additional keyboard layouts (AZERTY, QWERTZ).
- Periodically refresh the dictionary/common-password set from updated public breachcompilation sources.
- Benchmark the module against Module 3's full pipeline under production-scale concurrent load.

5.3 Professional Learning & Individual Reflection New

Knowledge Acquired:

- Practical experience applying run-length encoding and sliding-window techniques to real-world string analysis.
- Understanding of NIST and OWASP guidance on password validation and how it translates into concrete engineering requirements.

- Experience structuring a weighted decision matrix to justify a design choice against measurable alternatives.

Engineering & Professional Skills Developed:

- Algorithm design and complexity analysis under explicit latency constraints.
- Test-driven development practice using unit tests to validate detector behaviour before integration.
- Technical report writing and structured documentation of engineering trade-offs.

Individual Reflection:

The most difficult engineering problem I encountered was deciding how to catch overlapping pattern types — a password that is simultaneously a keyboard sequence and a dictionary word with substitutions — without over-complicating the detectors or increasing false positives. I resolved it by keeping the three detectors independent and explainable, and explicitly logging the mixed-pattern gap as future work supported by test evidence (Table 12) rather than adding untested complexity under time pressure. The key engineering decision I made personally was choosing the hybrid regex/hash-set design over the deep-learning alternative; the evidence in the weighted evaluation matrix (Table 6) and the measured 2.1 ms latency confirmed this was the right trade-off for the module's non-functional requirements. Independently, I learned how national and industry security guidelines (NIST SP 800-63B, OWASP) translate into concrete, testable software requirements rather than abstract advice. If I were to redesign the module, I would build the substitution-normalisation step in from the start rather than treating it as an afterthought, since the test results show it is the single largest source of residual detection error.

REFERENCES

[1] Buyya, R., Yeo, C. S., Venugopal, S., Broberg, J., and Brandic, I., “Cloud computing and emerging IT platforms

[2] Beloglazov, A., Abawajy, J., and Buyya, R., “Energy-aware resource allocation heuristics for efficient managementof data centers for Cloud computing,” Future Generation Computer Systems.

[3] Beloglazov, A. and Buyya, R., “Optimal online deterministic algorithms and adaptive heuristics for energy andperformance efficient dynamic consolidation of virtual machines in Cloud data centers,” Concurrency and Computation: Practice and Experience.

[4] Deb, K., Pratap, A., Agarwal, S., and Meyarivan, T., “A fast and elitist multiobjective genetic algorithm: NSGA-II,”IEEE Transactions on Evolutionary Computation.

[5] Green Software Foundation, “Green Software Principles,” guidance on software energy and carbon efficiency.

[6] International Energy Agency, “Data centres and data transmission networks,” analysis of electricity demand anddigital infrastructure.

[7] Buyya, R., Beloglazov, A., and Abawajy, J., “Energy-efficient management of data center resources for cloudcomputing,” selected research on dynamic consolidation and energy-aware management.

[8] Garg, S. K., Yeo, C. S., Anandasivam, A., and Buyya, R., research on SLA-oriented resource allocation andenergy-aware cloud management.

[9] Python Software Foundation, “Python 3 Documentation,” language and standard-library reference.

[10] NumPy Developers, “NumPy Documentation,” numerical computing reference.

[11] pandas Development Team, “pandas Documentation,” data analysis and processing reference.

[12] Matplotlib Development Team, “Matplotlib Documentation,” scientific visualisation reference.

APPENDICES

Appendix A – Detailed Calculations

For workload i, let P represent average power in kW, t represent execution time in hours, and CI represent carbon intensity in gCO■e/kWh. Estimated energy is E = P × t. Estimated carbon is then C = E × CI. The schedule-level carbon objective is the sum of C across all workloads.

For monetary cost, if r is the resource price per unit-hour, then Cost = r × t. A schedule with lower carbon but higher cost is not automatically inferior; it may be non-dominated and therefore retained as a Pareto alternative.

Appendix B – Engineering Drawings / Schematics

Figure 1 in Chapter 3 provides the architecture. Figure 2 presents the control flow from workload input to candidate generation, Pareto optimisation, local search and final schedule selection.

Appendix C – Source Code / Code Repository Information

Representative implementation structure:

Module	Purpose
workload_model.py	Defines workload and resource data structures.
carbon_model.py	Stores carbon-intensity and price profiles.
heuristic_scheduler.py	Generates feasible carbon-aware candidate schedules.
pareto_optimizer.py	Performs non-dominated sorting and candidate selection.
local_search.py	Applies swap, shift and migration improvements.
evaluation.py	Calculates carbon, cost, makespan and SLA metrics.
main.py	Runs experiments and exports results.

Appendix D – Experimental Parameters

Parameter	Value Used in Representative Experiment
Carbon intensity range	100–700 gCO■e/kWh
Resource price range	■1.5–■6.0 per resource-hour
Workload flexibility	0–12 time slots
Deadline strictness	Loose / medium / tight
Candidate pool	50–200 feasible candidates per run
Local-search neighbourhood	Time shift, resource swap, migration

Appendix E – Experimental / Raw Data

The final project submission should attach the generated workload instances, carbon-intensity profiles, price profiles, random seeds and per-run scheduler metrics. The redesigned report does not claim that those raw files were present in the uploaded source document.

Appendix F – Ethics / Institutional Approval

This project is a software-based simulation study. It does not directly involve human participants, clinical data, or physical experiments. Any future use of operational cloud telemetry should apply appropriate access control, anonymisation and institutional data-governance requirements.